Práctica 5- INTEGRACIÓN, AUTOMATIZACIÓN Y MODELO DE NEGOCIO

TRABAJO REALIZADO POR:
JOSÉ A. RODRÍGUEZ LÓPEZ

Contenido

API de Generación de Texto con Proxy Seguro 5

1. Introducción 5

2. Definiciones clave 5

2.1. FastAPI 5

2.2. Proxy 5

2.3. Generación de texto 5

2.4. Integración web 6

2.5. IA remota 6

2.6. Arquitectura cliente-servidor 6

3. Objetivo 6

3.1. Objetivos específicos 6

4. Arquitectura 7

5. Fases de desarrollo 7

6. Descripción técnica del sistema 8

6.1. Estructura de archivos 8

6.2. Configuración del entorno 8

6.3. Seguridad y políticas HTTP 8

6.4. Modelos y validación 9

6.5. Endpoints disponibles 10

6.6. Lógica de generación (POST /generate-proxy) 10

6.7. Postprocesamiento de texto 11

6.8. Manejo de errores 11

6.9. Estructura HTML del frontend (web/index.html) 18

6.10. Estilos CSS (web/styles.css) 21

6.11. Lógica JavaScript (web/app.js) 31

7. Flujo funcional 36

8. Dependencias requeridas 37

8.1. fastapi 37

8.2. uvicorn 37

8.3. pydantic 37

8.4. httpx 37

9. Instrucciones de ejecución local 37

Informe final 39

1. Resumen general 39

2. Modelo de automatización 39

2.1. Proceso de negocio objetivo 39

2.2. Flujo automatizado propuesto 40

2.3. Beneficios esperados 40

2.4. Riesgos y mitigaciones 40

3. Modelo de negocio para startup IA 41

3.1. Alternativas evaluadas 41

3.2 Comparación 41

3.3. Propuesta recomendada 42

3.4. Segmentos de clientes 42

3.5. Métricas clave 42

4. Estrategia corporativa (empresa existente) 43

4.1. Integración para ventaja competitiva 43

4.2. Ruta de adopción recomendada 43

4.3. Ventaja competitiva concreta 43

5. Convergencia tecnológica 44

5.1. IoT + IA 44

5.2. Blockchain + IA 44

6. Viabilidad y hoja de ruta 45

6.1. Viabilidad técnica 45

6.2. Viabilidad de negocio 45

6.3. Roadmap sugerido 45

7. Conclusión 46

API de Generación de Texto con Proxy Seguro

1. Introducción

Esta práctica desarrolla una aplicación web que genera texto mediante un proxy API. El sistema recibe parámetros del usuario, los envía a un servicio remoto, procesa el resultado y lo devuelve de forma limpia y coherente. El nivel de sofisticación radica en la validación exhaustiva, protección de credenciales y estrategia inteligente de ampliación de generación hasta obtener texto con cierre natural.

2. Definiciones clave

2.1. FastAPI

Framework moderno de Python pensado para crear APIs de forma rápida y ordenada. Destaca por su rendimiento, su soporte nativo para asincronía y por generar documentación automática. Es muy útil para montar servicios que expongan modelos de IA o microservicios ligeros.

2.2. Proxy

Servicio que actúa como intermediario entre un cliente y un servidor. Puede redirigir peticiones, aplicar filtros, añadir seguridad o unificar el acceso a varios servicios. En entornos de IA suele utilizarse para gestionar el tráfico hacia modelos remotos o proveedores externos.

2.3. Generación de texto

Capacidad de un modelo de inteligencia artificial para producir texto coherente a partir de una instrucción o contexto.

Incluye tareas como redactar, resumir, responder preguntas o continuar una frase. Los modelos LLM son los que realizan este tipo de trabajo.

2.4. Integración web

Conjunto de mecanismos que permiten que distintas aplicaciones o servicios se comuniquen entre sí a través de Internet.

Puede incluir APIs REST, WebSockets, webhooks, autenticación OAuth2, etc. Es la base para conectar un frontend con un backend o un backend con un servicio de IA.

2.5. IA remota

Modelo de inteligencia artificial que se ejecuta en un servidor externo, ya sea en la nube o en otra máquina.

El cliente envía la petición, el servidor procesa la solicitud y devuelve la respuesta. Esto permite usar modelos potentes sin necesidad de hardware local especializado.

2.6. Arquitectura cliente-servidor

Estructura clásica donde un cliente (navegador, aplicación, script) solicita un recurso o servicio, y un servidor lo procesa y responde.

Es la base de la mayoría de las aplicaciones web y APIs modernas, y encaja perfectamente con sistemas que integran IA remota.

3. Objetivo

El objetivo central es integrar un servicio remoto de generación de texto mediante un proxy API que actúe como capa de seguridad, control y normalización. En lugar de exponer el navegador directamente a la API remota, se implementa un servidor FastAPI encargado de validar solicitudes, proteger credenciales, aplicar políticas de calidad y gestionar la comunicación con el proveedor externo.

3.1. Objetivos específicos

- Implementar validación de entrada exhaustiva en el backend.

- Establecer comunicación segura y confiable con la API remota.

- Asegurar coherencia textual mediante ampliación incremental de generación hasta obtener cierre natural.

- Implementar medidas de seguridad HTTP (headers, protección de credenciales).

- Crear interfaz web responsiva que se adapte a múltiples dispositivos.

- Integrar funcionalidad de accesibilidad mediante síntesis de voz.

- Demostrar manejo robusto de errores y estados del sistema.

4. Arquitectura

La arquitectura se organiza en dos capas conectadas:

- **Capa de aplicación (Backend):** servidor FastAPI que valida, procesa y coordina con el servicio remoto.
- **Capa de presentación (Frontend):** interfaz web responsiva donde el usuario configura parámetros y consulta resultados.

Esta separación permite mantener control centralizado sobre seguridad, validación y normalización de datos.

5. Fases de desarrollo

1. Diseño de modelos de validación con Pydantic.

2. Implementación de endpoints de salud y configuración.

3. Desarrollo del proxy con estrategia incremental de generación.

4. Construcción de interfaz responsiva con layout adaptable.

5. Integración de lógica JavaScript.

6. Validación y pruebas de flujos.

6. Descripción técnica del sistema

6.1. Estructura de archivos

- api_simple.py: API principal, rutas HTTP, validaciones y lógica de proxy.
- web/index.html: estructura de la interfaz de usuario.
- web/styles.css: estilos visuales, distribución de paneles y comportamiento responsivo.
- web/app.js: interacción de la interfaz con el backend y manejo de estado.
- web/circuito-bg.js: fondo animado en canvas.
- requirements.txt: dependencias del entorno Python.

6.2. Configuración del entorno

El backend requiere dos variables de entorno:

- REMOTE_API_BASE_URL: dirección del servicio remoto.
- REMOTE_API_KEY: credencial de autenticación.

Si alguna está ausente, el servidor rechaza operaciones remotas y devuelve un error de configuración.

6.3. Seguridad y políticas HTTP

Cada respuesta del servidor incluye cabeceras de seguridad:

- X-Content-Type-Options: nosniff → previene interpretación errada de tipos MIME.
- X-Frame-Options: DENY → evita inyección en iframes.
- X-XSS-Protection: 1; mode=block → protección contra inyecciones XSS.
- Cache-Control: no-store → previene almacenamiento en caché.

También se habilita CORS para permitir comunicación del frontend con el backend.

6.4. Modelos y validación

Se utilizan modelos Pydantic:

- GenerateRequest
- GenerateResponse

Restricciones de validación:

- seed: 1 a 500 caracteres. El *seed* es la chispa inicial, el punto de partida que marca el tono de la generación. Puede ser una frase, una idea o incluso una palabra suelta. Con ella, el modelo entiende hacia dónde debe dirigirse: si debe continuar un pensamiento, imitar un estilo o simplemente expandir un concepto.

Es, en esencia, la primera piedra del camino narrativo.

- n_words: 1 a 200. Este parámetro actúa como una especie de "mínimo vital" para la respuesta. No obliga al modelo a detenerse exactamente en un número, pero sí le indica que debe desarrollar la idea con cierta amplitud.

Es como pedirle: *"No te quedes corto; dame algo con cuerpo, con sustancia."*

- strategy: greedy, sampling o top_k. Aquí decides **cómo** piensa el modelo mientras escribe.

greedy es el camino recto: siempre elige la palabra más probable, sin desviarse. Produce textos precisos, pero menos sorprendentes.

sampling abre la puerta a la espontaneidad: el modelo elige entre varias opciones posibles, lo que da lugar a textos más vivos y creativos.

top_k es un punto intermedio: limita la elección a un conjunto de palabras razonables, evitando tanto la rigidez como el caos.

Es, en cierto modo, elegir la personalidad del narrador.

- temperature: 0.1 a 2.0. La temperatura es el termostato de la creatividad.

Con valores bajos, el modelo se comporta como un escritor meticuloso, casi matemático.

Con valores altos, se vuelve más atrevido, más imaginativo, más dispuesto a explorar caminos inesperados.
Es la diferencia entre un informe técnico y un poema improvisado.

- top_k: 1 a 500. Cuando se usa la estrategia *top_k*, este parámetro define cuántas opciones tiene el modelo sobre la mesa en cada paso.

Un *top_k* pequeño lo mantiene disciplinado; uno grande le permite divagar, probar combinaciones nuevas, arriesgarse un poco más.
Es como decidir si el escritor tiene un vocabulario limitado o un abanico amplio de posibilidades.

Solicitudes fuera de estos límites son rechazadas con respuesta de validación clara.

6.5. Endpoints disponibles

- GET /: información general de la API.
- GET /app: entrega la aplicación web.
- GET /favicon.ico: entrega el favicon.
- GET /health: salud del proxy y estado de configuración remota.
- GET /web-config: configuración mínima para inicializar el frontend.
- GET /health-remote: comprobación de conectividad con la API remota.
- POST /generate-proxy: generación de texto mediante proxy.

6.6. Lógica de generación (POST /generate-proxy)

El parámetro n_words define el mínimo inicial. El proxy implementa un algoritmo iterativo:

1. Realiza solicitud inicial a la API remota con n_words.
2. Si la respuesta no contiene punto (.), repite aumentando n_words en bloques de 20.
3. Continúa iterando hasta encontrar punto o alcanzar límite de 200 palabras.
4. Cuando detecta punto, recorta al último punto encontrado.

5. Si no hay punto antes del límite, devuelve la mejor salida obtenida.
6. Acumula tiempo total de todos los intentos en elapsed_ms.

Esta estrategia asegura coherencia textual sin bloqueos indefinidos.

6.7. Postprocesamiento de texto

Antes de la respuesta, el sistema normaliza el texto:

- Elimina tokens especiales de la IA (`<UNK>`, `<PAD>`, `<BOS>`, `<EOS>`).
- Corrige espaciado antes de puntuación.
- Normaliza mayúsculas (inicio de texto y después de punto).
- Recorta al último punto si existe.

6.8. Manejo de errores

- Conectividad remota: `502` (fallo en comunicación con API remota).
- Configuración local: `500` (variables de entorno faltantes o inválidas).
- Validación de entrada: respuesta clara de Pydantic/FastAPI.

```
# -*- coding: utf-8 -*-
"""
              PRÁCTICA 5: Integración, automatización y modelo de negocio
                       API de Generación de Texto con FastAPI
                       Programación de Inteligencia Artificial

Implementa una API RESTful con FastAPI que actúe como proxy hacia la API de
generación de texto desarrollada en la práctica anterior. La API debe exponer un
endpoint POST /generate-proxy que reciba los mismos parámetros que la API remota,
los valide y los reenvíe a la API remota, devolviendo la respuesta al cliente.
Además, implementa un endpoint GET /health-remote para verificar la conectividad
con la API remota. Asegúrate de manejar adecuadamente los errores y de incluir
medidas de seguridad como autenticación por API Key y limitación de tasa para
proteger el servicio.

DESARROLLADO POR:   José A. Rodríguez López
FECHA: 29 de Marzo, 2026
PROYECTO: Programación de Inteligencia Artificial
==============================================================================
"""
```

```python
from fastapi import FastAPI, HTTPException, Request # Librería principal para
crear la API y manejar excepciones
from fastapi.middleware.cors import CORSMiddleware # Librería para manejar CORS
from fastapi.responses import FileResponse # Librería para servir archivos
estáticos
from fastapi.staticfiles import StaticFiles # Librería para servir archivos
estáticos desde un directorio
from pydantic import BaseModel, Field, field_validator # Librería para validación
de datos y modelos
from typing import Literal # Librería para tipos literales en validación de datos
import httpx # Librería para realizar solicitudes HTTP asíncronas
import os # Librería para manejar variables de entorno y operaciones del sistema
import re # Librería para expresiones regulares, utilizada para limpiar el texto
generado

MAX_PROXY_WORDS = 200
PROXY_WORDS_STEP = 20

# Configuración de la API remota a través de variables de entorno, con valores por
defecto para desarrollo local
REMOTE_API_BASE_URL = os.getenv(
    "REMOTE_API_BASE_URL",
    "https://ca-practica4.wittycoast-99cc6956.westeurope.azurecontainerapps.io",
).rstrip("/")
REMOTE_API_KEY = os.getenv("REMOTE_API_KEY", "")
WEB_DIR = "web"
INDEX_FILE = os.path.join(WEB_DIR, "index.html")

# Configuración de la aplicación FastAPI y middleware para CORS y seguridad
app = FastAPI(title="Practica 5 Integracion", version="1.0.0")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Monta el directorio de archivos estáticos si existe, para servir el frontend
desde /static
if os.path.isdir(WEB_DIR):
    app.mount("/static", StaticFiles(directory=WEB_DIR), name="static")

# Middleware para agregar encabezados de seguridad a todas las respuestas HTTP
@app.middleware("http")
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
```

```
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-XSS-Protection"] = "1; mode=block"
    response.headers["Cache-Control"] = "no-store"
    return response

# Modelos de datos para la solicitud y respuesta del endpoint de generación de
texto, con validación de campos
class GenerateRequest(BaseModel):
    seed: str = Field(..., min_length=1, max_length=500)
    n_words: int = Field(default=50, ge=1, le=200)
    strategy: Literal["greedy", "sampling", "top_k"] = "sampling"
    temperature: float = Field(default=1.0, ge=0.1, le=2.0)
    top_k: int = Field(default=40, ge=1, le=500)

    @field_validator("strategy")
    @classmethod
    def validate_strategy(cls, value: str) -> str:
        allowed = {"greedy", "sampling", "top_k"}
        if value not in allowed:
            raise ValueError(f"La estrategia debe ser una de {allowed}")
        return value


class GenerateResponse(BaseModel):
    seed: str
    generated_text: str
    strategy: str
    n_words_generated: int
    elapsed_ms: float


def _require_remote_config() -> tuple[str, str]:
    """
    Verifica que la configuración de la API remota esté presente.
    Returns:
        Tuple con la URL base de la API remota y la clave de API.
    Raises:
        HTTPException con código 500 si falta alguna configuración.
    """
    if not REMOTE_API_BASE_URL:
        raise HTTPException(status_code=500, detail="REMOTE_API_BASE_URL no
configurada")
    if not REMOTE_API_KEY:
        raise HTTPException(status_code=500, detail="REMOTE_API_KEY no
configurada")
    return REMOTE_API_BASE_URL, REMOTE_API_KEY
```

```python
def _clean_generated_text(text: str) -> str:
    """
    Elimina tokens especiales visibles del texto generado remoto.
    Args:
        text: El texto generado que puede contener tokens como <UNK>, <PAD>,
<BOS>, <EOS>.
    Returns:
        El texto limpio, con tokens eliminados y formato mejorado.
    """
    if not text:
        return text
    cleaned = re.sub(r"</?(UNK|PAD|BOS|EOS)>", " ", text, flags=re.IGNORECASE)
    cleaned = re.sub(r"\b(UNK|PAD|BOS|EOS)\b", " ", cleaned, flags=re.IGNORECASE)
    cleaned = re.sub(r"\s+([,.;:!?])", r"\1", cleaned)
    cleaned = re.sub(r"\s{2,}", " ", cleaned).strip()

    # Capitaliza el inicio del texto y la primera letra tras fin de oración.
    chars = list(cleaned)
    capitalize_next = True
    for i, ch in enumerate(chars):
        if capitalize_next and ch.isalpha():
            chars[i] = ch.upper()
            capitalize_next = False
        if ch in ".!?":
            capitalize_next = True
    return "".join(chars)


def _trim_to_last_period(text: str) -> str:
    """
    Si el texto no termina en punto, recorta hasta el último punto encontrado.
    Si no hay puntos, devuelve el texto limpio sin modificar.
    """
    cleaned = (text or "").strip()
    if not cleaned or cleaned.endswith("."):
        return cleaned

    last_period = cleaned.rfind(".")
    if last_period == -1:
        return cleaned
    return cleaned[: last_period + 1].strip()


def _count_words(text: str) -> int:
    """
```

```
    Cuenta palabras no vacías en un texto.
    """
    return len(re.findall(r"\S+", text or ""))


@app.get("/")
async def root():
    """
    Endpoint raíz que proporciona información básica sobre la API y enlaces a la
documentación y aplicación web.
    Returns:
        Un diccionario con un mensaje de bienvenida y enlaces a la documentación y
aplicación.
    """
    return {"message": "Practica 5 Integracion", "docs": "/docs", "app": "/app"}


@app.get("/app")
async def web_app():
    """
    Endpoint que sirve la aplicación web desde el archivo index.html.
    Returns:
        FileResponse con el archivo index.html si existe, HTTPException 404 si no.
    """
    if not os.path.exists(INDEX_FILE):
        raise HTTPException(status_code=404, detail="Frontend no encontrado en
/web")
    return FileResponse(INDEX_FILE)


@app.get("/favicon.ico", include_in_schema=False)
async def favicon():
    """
    Endpoint que sirve el favicon.ico desde el directorio web.
    Returns:
        FileResponse con el archivo favicon.ico si existe, HTTPException 404 si
no.
    """
    favicon_path = os.path.join(WEB_DIR, "favicon.ico")
    if os.path.exists(favicon_path):
        return FileResponse(favicon_path)
    raise HTTPException(status_code=404, detail="favicon.ico no encontrado")


@app.get("/health")
async def health():
    """
```

```
    Endpoint de salud que verifica la configuración de la API remota y devuelve
información básica sobre el servicio.
    Returns:
        Un diccionario con el estado del servicio, la URL base de la API remota y
si la clave remota está configurada.
    """
    return {
        "status": "ok",
        "service": "practica5-proxy",
        "remote_api_base_url": REMOTE_API_BASE_URL,
        "remote_key_configured": bool(REMOTE_API_KEY),
    }


@app.get("/web-config")
async def web_config():
    """
    Endpoint que proporciona la configuración de la API remota para el frontend.
    Returns:
        Un diccionario con la configuración de la API remota para el frontend.
    """
    return {
        "mode": "proxy",
        "remote_api_base_url": REMOTE_API_BASE_URL,
        "remote_key_configured": bool(REMOTE_API_KEY),
    }


@app.get("/health-remote")
async def health_remote():
    """
    Endpoint que verifica la conectividad con la API remota realizando una
solicitud al endpoint de salud remoto.
    Returns:
        Un diccionario con el estado de la API remota si la conexión es exitosa.
    Raises:
        HTTPException con código 502 si hay un error al conectar con la API
remota.
    """
    base_url, _ = _require_remote_config()
    try:
        async with httpx.AsyncClient(timeout=20.0) as client:
            response = await client.get(f"{base_url}/health")
        response.raise_for_status()
        return {"status": "ok", "remote": response.json()}
    except httpx.HTTPError as exc:
```

```
        raise HTTPException(status_code=502, detail=f"Error conectando con API
remota. {exc}") from exc


@app.post("/generate-proxy", response_model=GenerateResponse)
async def generate_proxy(body: GenerateRequest):
    """
    Endpoint que actúa como proxy para la generación de texto utilizando la API
remota.
    Args:
        body: Una instancia de GenerateRequest con los parámetros de generación.
    Returns:
        Una instancia de GenerateResponse con el texto generado.
    Raises:
        HTTPException con código 502 si hay un error al conectar con la API
remota.
    """
    base_url, remote_key = _require_remote_config()
    headers = {"Content-Type": "application/json", "X-API-Key": remote_key}
    try:
        target_words = body.n_words
        total_elapsed_ms = 0.0
        last_data: dict | None = None

        async with httpx.AsyncClient(timeout=60.0) as client:
            while target_words <= MAX_PROXY_WORDS:
                payload = body.model_dump()
                payload["n_words"] = target_words

                response = await client.post(
                    f"{base_url}/generate",
                    json=payload,
                    headers=headers,
                )

                if response.status_code >= 400:
                    detail = response.json().get("detail", f"HTTP
{response.status_code}")
                    raise HTTPException(status_code=response.status_code,
detail=detail)

                data = response.json()
                total_elapsed_ms += float(data.get("elapsed_ms", 0.0) or 0.0)

                cleaned_text = _clean_generated_text(data.get("generated_text",
""))
                data["generated_text"] = cleaned_text
```

```
                data["elapsed_ms"] = total_elapsed_ms
                last_data = data

                # Si ya hay al menos un punto, se recorta hasta el último punto y
se devuelve.
                if "." in cleaned_text:
                    data["generated_text"] = _trim_to_last_period(cleaned_text)
                    data["n_words_generated"] =
_count_words(data["generated_text"])
                    return GenerateResponse(**data)

                target_words += PROXY_WORDS_STEP

        # Si no se encontró punto dentro del límite, se devuelve la mejor salida
disponible.
        if last_data is None:
            raise HTTPException(status_code=502, detail="No se recibió respuesta
válida de la API remota")

        last_data["n_words_generated"] =
_count_words(last_data.get("generated_text", ""))
        return GenerateResponse(**last_data)
    except HTTPException:
        raise
    except httpx.HTTPError as exc:
        raise HTTPException(status_code=502, detail=f"Error de proxy hacia API
remota. {exc}") from exc
```

6.9. Estructura HTML del frontend (web/index.html)

La interfaz presenta:

- Título identificador del sistema.
- Formulario para semilla de texto y configuración de parámetros.
- Bloque de información del endpoint remoto.
- Área de salida para texto generado y tiempo de respuesta.
- Controles: generar, reproducir (voz), detener (voz).

```
<!--
              PRÁCTICA 5: Integración, automatización y modelo de negocio
        Interfaz de usuario para generación de texto con modelo generativo
                     Programación de Inteligencia Artificial
```

```
Este archivo HTML es la interfaz de usuario para la práctica 5, diseñada para
interactuar con un modelo generativo a través de un endpoint remoto. La estructura
del documento incluye un formulario para configurar los parámetros de generación,
un área para mostrar el texto generado, y elementos para mostrar el estado y el
tiempo transcurrido.

DESARROLLADO POR:   José A. Rodríguez López
FECHA: 29 de Marzo, 2026
PROYECTO: Programación de Inteligencia Artificial
==============================================================================
-->

<!doctype html>
<html lang="es">
  <head>
    <!-- Metadatos base y recursos de estilo global -->
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Práctica 5 | IA Integrada</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;70
0&family=IBM+Plex+Mono:wght@400;500&display=swap"
      rel="stylesheet"
    />
    <link rel="icon" type="image/x-icon" href="/static/favicon.ico" />
    <link rel="stylesheet" href="/static/styles.css" />
  </head>
  <body>
    <!-- Fondo animado decorativo -->
    <canvas id="fondoCircuito"></canvas>

    <!-- Contenido principal de la interfaz -->
    <main class="contenedor-principal">
      <!-- Encabezado de la práctica -->
      <section class="cabecera">
        <h1>Integración tecnológica con modelo generativo</h1>
      </section>

      <!-- Panel de entrada: configuración y envío de generación -->
      <section class="panel panel-principal">
        <form id="formulario-generador" class="formulario-cuadricula">
          <p class="resumen-endpoint">
            Endpoint remoto: <strong id="urlApiRemota">cargando...</strong>
          </p>
```

```
          <div class="acciones">
            <button id="botonConexion" type="button" class="boton-
secundario">Probar conexión remota</button>
          </div>

          <label>
            Semilla de texto
            <textarea
              id="semilla"
              rows="4"
              placeholder="Ejemplo: En una ciudad inteligente, los sensores
detectaron..."
              required
            ></textarea>
          </label>

          <div class="fila dos-columnas">
            <label>
              Número de palabras
              <input id="numeroPalabras" type="number" min="1" max="100"
value="40" />
            </label>

            <label>
              Estrategia
              <select id="estrategia">
                <option value="greedy">Greedy</option>
                <option value="sampling">Sampling</option>
                <option value="top_k">Top-K</option>
              </select>
            </label>
          </div>

          <div class="fila dos-columnas">
            <label>
              Temperatura
              <input id="temperatura" type="number" min="0.1" max="2" step="0.1"
value="1.0" />
            </label>

            <label>
              Top-K
              <input id="topK" type="number" min="1" max="500" value="40" />
            </label>
          </div>

          <div class="acciones">
```

```
            <button id="botonGenerar" type="submit">Generar texto</button>
            <button id="botonHablar" type="button" class="boton-secundario">Hablar
resultado</button>
            <button id="botonDetenerVoz" type="button" class="boton-
secundario">Detener voz</button>
          </div>
        </form>
      </section>

      <!-- Panel de salida: estado y texto generado -->
      <section class="panel panel-secundario contenedor-salida">
        <div class="metadatos-estado">
          <span id="estado">Listo</span>
          <span id="tiempoTranscurrido"></span>
        </div>
        <article id="salidaGenerada" class="salida-generada" aria-live="polite">
          El texto generado aparecerá aquí.
        </article>
      </section>
    </main>

    <!-- Pie de página con créditos -->
    <footer class="pie-sitio">
        <p>Desarrollado por: José A. Rodríguez López</p>
        <p class="texto-destacado">Programación de IA · Práctica 5</p>
    </footer>

    <!-- Scripts de animación de fondo y lógica de interacción -->
    <script src="/static/circuito-bg.js"></script>
    <script src="/static/app.js"></script>
  </body>
</html>
```

6.10. Estilos CSS (web/styles.css)

Diseño visual e implementación:

- Paleta de colores definida mediante variables CSS en español.
- Layout en dos columnas en desktop (formulario e información remota a la izquierda, salida a la derecha).
- Disposición de una columna en dispositivos móviles.
- Paneles con sombras y bordes redondeados.
- Fondo con gradientes radiales y animación complementaria en canvas.

```css
/* Variables de color y estilos globales para la aplicación web. */
:root {
  --tinta-hero: #333333;
  --tinta: #333333;
  --tinta-suave: rgba(51, 51, 51, 0.62);
  --acento: #1A73E8;
  --acento-fuerte: #1558B0;
  --acento-secundario: #4CAF50;
  --papel: rgba(247, 247, 247, 0.96);
  --papel-fuerte: rgba(247, 247, 247, 0.98);
  --papel-suave: rgba(243, 243, 243, 0.96);
  --fondo-salida: rgba(247, 247, 247, 0.95);
  --borde-panel: rgba(51, 51, 51, 0.08);
  --linea: rgba(51, 51, 51, 0.12);
  --campo: #E3E3E3;
}

/* Estilos globales para la aplicación web. */
* {
  box-sizing: border-box;
}

/* Estilos para el cuerpo y la estructura general de la página. */
html {
  height: 100%;
}

/* Estilos para el cuerpo de la página, incluyendo la fuente, colores y fondo con
gradientes radiales. */
body {
  margin: 0;
  min-height: 100vh;
  min-height: 100dvh;
  font-family: "Space Grotesk", sans-serif;
  color: #333333;
  display: flex;
  flex-direction: column;
  background:
    radial-gradient(circle at 12% 22%, rgba(162, 166, 12, 0.22), transparent 36%),
    radial-gradient(circle at 32% 14%, rgba(12, 166, 47, 0.24), transparent 40%),
    radial-gradient(circle at 56% 18%, rgba(12, 100, 166, 0.26), transparent 44%),
    radial-gradient(circle at 76% 20%, rgba(108, 12, 166, 0.24), transparent 42%),
    radial-gradient(circle at 92% 28%, rgba(166, 12, 39, 0.22), transparent 38%),
    radial-gradient(circle at 50% 85%, rgba(50, 166, 100, 0.16), transparent 42%),
    linear-gradient(160deg, #070b1f 0%, #0c1330 50%, #11133a 100%);
}
```

```
/* Estilos para el lienzo de fondo y la superposición de contenido. */
canvas {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 0;
  opacity: 0.24;
}

/* Estilos para el contenedor principal y el pie del sitio, incluyendo la posición
relativa para permitir la superposición de elementos. */
.contenedor-principal,
.pie-sitio {
  position: relative;
  z-index: 1;
}

/* Estilos para el contenedor principal, utilizando una cuadrícula para organizar
la cabecera, el contenido principal y el secundario, con un fondo difuminado y
animación de aparición. */
.contenedor-principal {
  width: min(1240px, 96vw);
  margin: 17px auto 20px auto;
  display: grid;
  grid-template-columns: minmax(0, 1.45fr) minmax(300px, 0.75fr);
  grid-template-areas:
    "cabecera cabecera"
    "principal secundario";
  gap: 20px;
  flex: 1;
  min-height: 0;
  animation: rise 500ms ease-out;
  isolation: isolate;
}

/* Estilos para la superposición difuminada detrás del contenido principal,
utilizando un gradiente radial y un filtro de desenfoque para crear un efecto
visual atractivo. */
.contenedor-principal::before {
  content: "";
  position: absolute;
  inset: 57px -18px 116px;
  z-index: -1;
  border-radius: 32px;
  background: radial-gradient(circle at 50% 32%, rgba(255, 255, 255, 0.08),
rgba(26, 115, 232, 0.02) 62%, transparent 78%);
  filter: blur(20px);
```

```
}

/* Estilos para la cabecera, organizando el contenido en una columna centrada con
espacio entre los elementos, y estilos específicos para el título y el texto
destacado. */
.cabecera {
  display: flex;
  flex-direction: column;
  align-items: center;
  text-align: center;
  gap: 12px;
  grid-area: cabecera;
}

/* Estilos para el título principal en la cabecera, utilizando un tamaño de fuente
adaptable con clamp, un color claro con sombra para mejorar la legibilidad sobre
el fondo, y un espaciado entre letras para un efecto visual más impactante. */
.cabecera h1 {
  margin: 8px 0;
  font-size: clamp(1.8rem, 4vw, 3rem);
  line-height: 1.1;
  letter-spacing: -0.02em;
  color: #F7F7F7;
  text-shadow: 0 6px 24px rgba(2, 10, 20, 0.52);
}

/* Estilos para el texto destacado en la cabecera, utilizando una fuente
monoespaciada para un aspecto técnico, un tamaño de fuente más pequeño, y un
espaciado entre letras para resaltar el texto de manera sutil. */
.texto-destacado {
  margin: 0;
  font-family: "IBM Plex Mono", monospace;
  font-size: 0.8rem;
  text-transform: uppercase;
  letter-spacing: 0.12em;
  opacity: 0.78;
}

/* Estilos para el resumen del endpoint, organizando el contenido en una fila con
espacio entre los elementos, y utilizando estilos para manejar el desbordamiento
horizontal de manera elegante. */
.resumen-endpoint {
  margin: 0;
  display: flex;
  align-items: center;
  gap: 0.35rem;
  min-width: 0;
```

```
  flex-wrap: nowrap;
  font-weight: 500;
  color: #333333;
  overflow-x: auto;
  overflow-y: hidden;
  white-space: nowrap;
  scrollbar-width: thin;
}

/* Estilos para el campo de URL de la API remota, asegurando que no se reduzca
demasiado en pantallas pequeñas y que mantenga un tamaño adecuado para su
contenido. */
#urlApiRemota {
  min-width: 0;
  flex: 0 0 auto;
}

/* Estilos para los paneles de contenido, utilizando colores y sombras para crear
un efecto de profundidad, bordes redondeados para un aspecto moderno, y un fondo
con un efecto de papel difuminado. */
.panel {
  color: var(--tinta);
  border: 1px solid var(--linea);
  border-radius: 20px;
  background: var(--papel);
  backdrop-filter: blur(6px);
  box-shadow: 0 18px 42px rgba(4, 14, 24, 0.18), 0 0 0 1px var(--borde-panel);
  padding: 18px;
  min-height: 0;
}

/* Estilos específicos para el panel principal, utilizando un fondo más fuerte y
una sombra más pronunciada para destacarlo como el área principal de interacción.
*/
.panel-principal {
  grid-area: principal;
  background: var(--papel-fuerte);
  border-color: rgba(26, 115, 232, 0.16);
  box-shadow: 0 28px 68px rgba(4, 14, 24, 0.24), 0 0 0 1px rgba(26, 115, 232,
0.08);
}

/* Estilos específicos para el panel secundario, utilizando un fondo más suave y
una sombra más ligera para diferenciarlo del panel principal, y organizando su
contenido en una columna. */
.panel-secundario {
  grid-area: secundario;
```

```
  background: var(--papel-suave);
  box-shadow: 0 12px 28px rgba(4, 14, 24, 0.14), 0 0 0 1px rgba(26, 115, 232,
0.06);
  display: flex;
  flex-direction: column;
}

/* Estilos para los formularios, organizando los campos en una cuadrícula con
espacio entre ellos, y estilos para las etiquetas y campos de entrada para mejorar
la usabilidad y la apariencia visual. */
.formulario-cuadricula {
  display: grid;
  gap: 12px;
}

/* Estilos para las etiquetas de los campos de formulario, organizando el
contenido en una cuadrícula con espacio entre el texto de la etiqueta y el campo
de entrada, y utilizando un tamaño de fuente ligeramente más pequeño para un
aspecto más compacto. */
label {
  display: grid;
  gap: 6px;
  font-size: 0.95rem;
}

/* Estilos para los campos de entrada, textarea, select y botones, asegurando que
hereden la fuente del cuerpo para mantener la coherencia visual, y estilos
específicos para los campos de entrada para mejorar su apariencia y usabilidad. */
input,
textarea,
select,
button {
  font: inherit;
}

/* Estilos para los campos de entrada, textarea y select, utilizando colores y
sombras para crear un efecto de profundidad, bordes redondeados para un aspecto
moderno, y un fondo con un efecto de campo difuminado. */
input,
textarea,
select {
  width: 100%;
  border: 1px solid var(--linea);
  border-radius: 12px;
  padding: 10px 12px;
  background: var(--campo);
  color: var(--tinta);
```

```
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.55);
}

/* Estilos para los placeholders de los campos de entrada y textarea, utilizando
un color más suave para diferenciar el texto de placeholder del texto ingresado
por el usuario. */
input::placeholder,
textarea::placeholder {
  color: rgba(51, 51, 51, 0.4);
}

textarea {
  resize: vertical;
  min-height: 96px;
  max-height: 18vh;
}

/* Estilos para las filas de contenido, organizando el contenido en una cuadrícula
con espacio entre los elementos, y estilos específicos para las filas de dos
columnas para crear una disposición equilibrada. */
.fila.dos-columnas {
  display: grid;
  gap: 12px;
  grid-template-columns: 1fr 1fr;
}

/* Estilos para las acciones, organizando los botones en una fila con espacio
entre ellos, permitiendo que se ajusten a múltiples líneas en pantallas pequeñas,
y centrando los botones para una apariencia más equilibrada. */
.acciones {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  justify-content: center;
}

/* Estilos para los botones, utilizando colores y sombras para crear un efecto de
profundidad, bordes redondeados para un aspecto moderno, y transiciones suaves
para mejorar la interactividad. */
button {
  border: none;
  border-radius: 999px;
  padding: 11px 18px;
  cursor: pointer;
  font-weight: 700;
  transition: transform 180ms ease, filter 180ms ease, box-shadow 180ms ease,
background-color 180ms ease;
```

```
}

/* Estilos para los botones al pasar el cursor sobre ellos, utilizando una ligera
transformación y ajuste de brillo para mejorar la interactividad visual. */
button:hover {
  transform: translateY(-2px);
  filter: brightness(0.99);
}

/* Estilos específicos para el botón de generación, utilizando un gradiente de
color para el fondo, un color claro para el texto, y una sombra más pronunciada
para destacarlo como la acción principal. */
#botonGenerar {
  background: linear-gradient(180deg, #1558B0 0%, #1A73E8 100%);
  color: #F7F7F7;
  letter-spacing: 0.02em;
  box-shadow: 0 14px 28px rgba(26, 115, 232, 0.28);
}

/* Estilos para el botón de generación al pasar el cursor sobre él, utilizando una
sombra más pronunciada para mejorar la interactividad visual. */
#botonGenerar:hover {
  box-shadow: 0 18px 36px rgba(26, 115, 232, 0.34);
}

/* Estilos específicos para el botón secundario, utilizando un fondo más suave y
una sombra más ligera para diferenciarlo del botón de generación, y un color de
texto más oscuro para mejorar la legibilidad. */
.boton-secundario {
  background: #F7F7F7;
  color: #333333;
  border: 1px solid #D0D0D0;
  box-shadow: 0 8px 18px rgba(8, 18, 28, 0.06);
}

/* Estilos para el botón secundario al pasar el cursor sobre él, utilizando una
sombra más pronunciada para mejorar la interactividad visual. */
.boton-secundario:hover {
  box-shadow: 0 12px 22px rgba(8, 18, 28, 0.10);
}

/* Estilos para el contenedor de salida, utilizando overflow hidden para manejar
el desbordamiento del contenido generado. */
.contenedor-salida {
  overflow: hidden;
}
```

```
/* Estilos para los metadatos del estado, organizando el contenido en una fila con
espacio entre los elementos, y utilizando una fuente monoespaciada para un aspecto
técnico. */
.metadatos-estado {
  display: flex;
  gap: 10px;
  font-family: "IBM Plex Mono", monospace;
  font-size: 0.85rem;
  color: var(--tinta-suave);
  margin-bottom: 12px;
}

/* Estilos para la salida generada, utilizando un borde de acento para destacarla,
un fondo con efecto de papel difuminado, y estilos para manejar el desbordamiento
del contenido generado de manera elegante. */
.salida-generada {
  margin: 0;
  border-left: 4px solid #4CAF50;
  padding: 12px 14px;
  border-radius: 10px;
  background: var(--fondo-salida);
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.65);
  min-height: 90px;
  flex: 1;
  overflow: auto;
  white-space: pre-wrap;
  line-height: 1.6;
  color: var(--tinta);
}

/* Estilos para el pie del sitio, organizando el contenido en una fila centrada, y
utilizando una fuente monoespaciada con un color suave para un aspecto técnico y
discreto. */
.pie-sitio {
  width: min(1240px, 96vw);
  margin: 0 auto 18px;
  text-align: center;
  font-family: "IBM Plex Mono", monospace;
  font-size: 0.85rem;
  color: rgba(247, 247, 247, 0.74);
}

/* Animación para la aparición de los elementos, utilizando una transición suave
de opacidad y posición para crear un efecto de elevación al aparecer. */
@keyframes rise {
  from {
    opacity: 0;
```

```
    transform: translateY(8px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Estilos para pantallas pequeñas, utilizando una disposición de bloque para el
cuerpo y reorganizando la cuadrícula del contenedor principal para apilar los
elementos verticalmente, con ajustes específicos para el resumen del endpoint, las
filas de dos columnas, las acciones y los botones para mejorar la usabilidad en
dispositivos móviles. */
@media (max-width: 720px) {
  body {
    display: block;
  }

  .contenedor-principal {
    grid-template-columns: 1fr;
    grid-template-areas:
      "cabecera"
      "principal"
      "secundario";
    margin-bottom: 24px;
  }

  .resumen-endpoint {
    font-size: 0.92rem;
  }

  .fila.dos-columnas {
    grid-template-columns: 1fr;
  }

  .acciones {
    flex-direction: column;
  }

  button {
    width: 100%;
  }

  textarea {
    max-height: none;
  }
}
```

6.11. Lógica JavaScript (web/app.js)

Gestiona la interacción del usuario:

- Al cargar, consulta configuración del backend y muestra endpoint remoto.
- Prueba conectividad remota cuando el usuario lo solicita.
- Envía solicitudes de generación al proxy con validación local.
- Presenta texto generado, tiempo acumulado y estado operativo.
- Maneja errores con mensajes informativos al usuario.
- Integra Text-to-Speech (español) para lectura del resultado.

```
/*
              PRÁCTICA 5: Integración, automatización y modelo de negocio
                Script de interacción con backend y generación de texto
                       Programación de Inteligencia Artificial

Este archivo contiene el código JavaScript para la aplicación web que interactúa
con el backend de FastAPI para generar texto utilizando un modelo de lenguaje. El
script maneja la carga de la configuración, la interacción con los elementos del
formulario, la comunicación con el backend a través de fetch, y la funcionalidad
de Text-to-Speech para leer el texto generado.

DESARROLLADO POR:   José A. Rodríguez López
FECHA: 29 de Marzo, 2026
PROYECTO: Programación de Inteligencia Artificial
==============================================================================
*/

// Elementos del DOM para interacción y visualización de resultados
const formulario = document.getElementById("formulario-generador");
const urlApiRemota = document.getElementById("urlApiRemota");
const salidaGenerada = document.getElementById("salidaGenerada");
const etiquetaEstado = document.getElementById("estado");
const etiquetaTiempo = document.getElementById("tiempoTranscurrido");
const botonConexion = document.getElementById("botonConexion");
const botonHablar = document.getElementById("botonHablar");
const botonDetenerVoz = document.getElementById("botonDetenerVoz");

// Variable para rastrear si la configuración remota está lista para usar
let configuracionRemotaLista = false;

// Función para actualizar el estado de la aplicación en la interfaz de usuario
const establecerEstado = (texto) => {
  etiquetaEstado.textContent = texto;
```

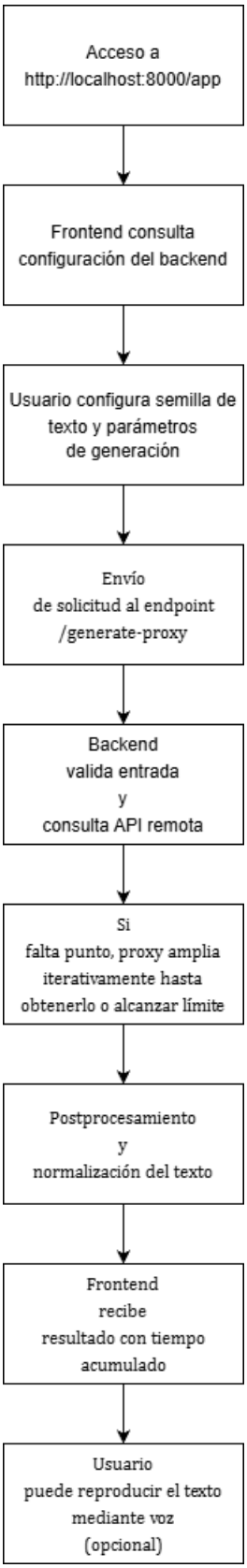
```
};

// Función para cargar la configuración del backend y actualizar la interfaz de
usuario en consecuencia
const cargarConfiguracionWeb = async () => {
  try {
    const respuesta = await fetch("/web-config");
    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}`);
    }
    const configuracion = await respuesta.json();
    urlApiRemota.textContent = configuracion.remote_api_base_url || "no
configurada";
    if (!configuracion.remote_key_configured) {
      establecerEstado("Falta REMOTE_API_KEY en el backend");
      configuracionRemotaLista = false;
      return;
    }
    configuracionRemotaLista = true;
    establecerEstado("Listo");
  } catch (error) {
    urlApiRemota.textContent = "error cargando configuración";
    establecerEstado(`Error de configuración: ${error.message}`);
    configuracionRemotaLista = false;
  }
};

// Cargar la configuración al iniciar la aplicación
cargarConfiguracionWeb();

// Evento para probar la conexión con el backend y la API remota, actualizando el
estado en la interfaz de usuario según el resultado
botonConexion.addEventListener("click", async () => {
  if (!configuracionRemotaLista) {
    establecerEstado("Configura primero REMOTE_API_BASE_URL y REMOTE_API_KEY en
backend");
    return;
  }

  establecerEstado("Probando conexión...");
  try {
    const respuesta = await fetch("/health-remote");
    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}`);
    }
    establecerEstado("Conexión con Azure API correcta");
  } catch (error) {
```

```
    establecerEstado(`No se pudo conectar: ${error.message}`);
  }
});

// Evento para manejar el envío del formulario de generación de texto, validando
los campos, enviando la solicitud al backend, y actualizando la interfaz de
usuario con el resultado o cualquier error que ocurra durante el proceso
formulario.addEventListener("submit", async (evento) => {
  evento.preventDefault();

  const semilla = document.getElementById("semilla").value.trim();
  const numeroPalabras = Number(document.getElementById("numeroPalabras").value);
  const estrategia = document.getElementById("estrategia").value;
  const temperatura = Number(document.getElementById("temperatura").value);
  const topK = Number(document.getElementById("topK").value);

  if (!configuracionRemotaLista) {
    establecerEstado("Configura primero REMOTE_API_BASE_URL y REMOTE_API_KEY en
backend");
    return;
  }
  if (!semilla) {
    establecerEstado("Faltan campos obligatorios");
    return;
  }

  const carga = {
    seed: semilla,
    n_words: numeroPalabras,
    strategy: estrategia,
    temperature: temperatura,
    top_k: topK,
  };

  establecerEstado("Generando...");
  etiquetaTiempo.textContent = "";

  try {
    const respuesta = await fetch("/generate-proxy", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(carga),
    });

    if (!respuesta.ok) {
```

```
      const error = await respuesta.json().catch(() => ({}));
      throw new Error(error.detail || `HTTP ${respuesta.status}`);
    }

    const datos = await respuesta.json();
    salidaGenerada.textContent = datos.generated_text;
    etiquetaTiempo.textContent = `${datos.elapsed_ms} ms`;
    establecerEstado("Texto generado correctamente");
  } catch (error) {
    salidaGenerada.textContent = "No se pudo generar texto.";
    establecerEstado(`Error: ${error.message}`);
  }
});

// Evento para manejar la funcionalidad de Text-to-Speech, verificando que haya
texto para leer, que el navegador soporte la síntesis de voz, y utilizando la API
de SpeechSynthesis para reproducir el texto generado, con actualizaciones de
estado durante el proceso de reproducción y manejo de errores
botonHablar.addEventListener("click", () => {
  const texto = salidaGenerada.textContent.trim();
  if (!texto || texto === "El texto generado aparecerá aquí.") {
    establecerEstado("No hay texto para leer");
    return;
  }

  if (!("speechSynthesis" in window)) {
    establecerEstado("Este navegador no soporta Text-to-Speech");
    return;
  }

  window.speechSynthesis.cancel();
  const locucion = new SpeechSynthesisUtterance(texto);
  locucion.lang = "es-ES";
  locucion.rate = 1;
  locucion.pitch = 1;

  locucion.onstart = () => establecerEstado("Reproduciendo voz...");
  locucion.onend = () => establecerEstado("Lectura finalizada");
  locucion.onerror = () => establecerEstado("Error al sintetizar voz");

  window.speechSynthesis.speak(locucion);
});

// Evento para detener la reproducción de voz en cualquier momento, utilizando la
API de SpeechSynthesis para cancelar cualquier locución en curso y actualizando el
estado en la interfaz de usuario para reflejar que la lectura ha sido detenida por
el usuario
```

```
botonDetenerVoz.addEventListener("click", () => {
  if ("speechSynthesis" in window) {
    window.speechSynthesis.cancel();
    establecerEstado("Lectura detenida");
  }
});
```

7. Flujo funcional



8. Dependencias requeridas

Definidas en requirements.txt:

8.1. fastapi

- Framework web moderno, asíncrono y muy rápido.
- Ideal para APIs REST, validación automática, documentación OpenAPI integrada.
- Encaja muy bien con estilo modular y reproducible.

8.2. uvicorn

- Servidor ASGI ultrarrápido.
- Es el motor que ejecuta FastAPI en producción o desarrollo.
- Ligero, estable y perfecto para contenedores Docker.

8.3. pydantic

- Base de la validación de datos en FastAPI.
- Define modelos estrictos y reproducibles.
- Muy útil para mantener claridad y evitar ambigüedades en los esquemas.

8.4. httpx

- Cliente HTTP asíncrono.
- Sustituye a requests cuando necesitas concurrencia real.
- Perfecto para microservicios que llaman a otros servicios o APIs externas.

9. Instrucciones de ejecución local

1. Configurar variables de entorno:

```
$env:REMOTE_API_BASE_URL="https://ca-practica4.wittycoast-
99cc6956.westeurope.azurecontainerapps.io"
$env:REMOTE_API_KEY="%KhJh-yj44k[RMuJpy"
```

2. Iniciar servidor:

```
uvicorn api_simple:app --host 0.0.0.0 --port 8000 --reload
```

3. Abrir en navegador:

```
http://localhost:8000/app
```

Informe final

1. Resumen general

Este proyecto integra una IA de generación de texto con una interfaz web y un backend proxy seguro para consumir una API desplegada en Azure. A partir de esta base técnica, se analiza su aplicación en procesos reales de negocio y su potencial como producto.

Conclusiones principales:

1. La IA permite automatizar procesos de redacción repetitivos, generando un impacto directo en la eficiencia y la productividad.
2. El modelo de entrada más sostenible es una estrategia freemium (versión gratuita básica que cobra por funciones avanzadas), con evolución natural hacia un esquema SaaS basado en planes (servicio en la nube que se ofrece mediante suscripciones con distintos niveles de funciones).
3. Las empresas pueden incorporar esta tecnología como un copiloto de contenido y apoyo a la decisión, reduciendo significativamente los tiempos de ciclo.
4. La integración con IoT (conjunto de dispositivos conectados a internet que recopilan y comparten datos para automatizar y optimizar procesos en tiempo real) y Blockchain (tecnología que permite registrar datos en una red distribuida de forma segura, transparente e inalterable) multiplica el valor añadido al habilitar trazabilidad, auditoría y generación automática de reportes.

2. Modelo de automatización

2.1. Proceso de negocio objetivo

Proceso seleccionado: generación de borradores para comunicación comercial (emails de marketing, descripciones de producto y publicaciones para redes).

Situación actual (sin IA):

1. Equipo de marketing redacta manualmente textos por canal.
2. Ciclo de aprobación lento entre redactor, responsable comercial y legal.
3. Alta variabilidad de calidad y tono.
4. Cuellos de botella en picos de demanda (lanzamientos o campañas).

Situación propuesta (con IA):

1. Usuario define semilla, objetivo y estilo.
2. La IA genera un primer borrador en segundos.
3. Revisor humano valida tono, marca y cumplimiento.
4. Publicación o envío en el canal correspondiente.

2.2. Flujo automatizado propuesto

1. Entrada: briefing corto (producto, publico, objetivo, tono).
2. Generación: llamada a la IA mediante API.
3. Postproceso: limpieza de tokens especiales y normalización básica de estilo.
4. revisión humana: control de calidad y compliance.
5. Salida: texto final para email, ecommerce o redes.

2.3. Beneficios esperados

1. Reducción del tiempo de redacción inicial entre 60% y 85%.
2. Mayor capacidad del equipo sin aumentar plantilla.
3. Estandarización de mensajes de marca.
4. Disminución del time-to-market en campañas.

2.4. Riesgos y mitigaciones

- **Riesgo 1**: textos con baja precisión factual.
- **Mitigación**: revisión humana obligatoria para piezas críticas.

- **Riesgo 2**: repeticiones o estilo poco natural.
- **Mitigación**: ajuste de estrategia de generación y plantillas por canal.

- **Riesgo 3**: dependencia del proveedor remoto.
- **Mitigación**: monitorización de latencia, fallback y cache de prompts comunes.

3. Modelo de negocio para startup IA

3.1. Alternativas evaluadas

1. **SaaS por suscripción:** ingresos recurrentes mediante planes mensuales o anuales con distintos niveles de servicio.
2. **Pago por uso (API calls):** el cliente paga únicamente por el volumen de consumo o llamadas realizadas.
3. **Freemium:** acceso gratuito básico con opciones avanzadas disponibles mediante pago.

3.2 Comparación

- **SaaS (suscripción)**

  Ventaja: genera ingresos recurrentes y predecibles.

  Desventaja: implica mayor fricción inicial en la decisión de compra.

- **Pago por uso**

  Ventaja: resulta especialmente atractivo para desarrolladores y equipos técnicos.

  Desventaja: produce ingresos variables y sensibles a la estacionalidad.

- **Freemium**

Ventaja: ofrece una barrera de entrada muy baja y permite un crecimiento rápido de usuarios.

Desventaja: exige controlar cuidadosamente los costes asociados al nivel gratuito.

3.3. Propuesta recomendada

Modelo recomendado: freemium + planes SaaS.

Estructura propuesta:

1. **Free**: límite mensual de generaciones y acceso únicamente a las funciones básicas.
2. **Pro**: mayor volumen de uso, longitudes ampliadas, soporte prioritario y analítica básica.
3. **Business**: funcionalidades para equipos, gestión de roles, auditoría, SLA y facturación anual.

3.4. Segmentos de clientes

1. Pymes de ecommerce.
2. Agencias de marketing y contenidos.
3. Equipos de comunicación en empresas medianas.
4. Startups que necesiten API de generación en sus productos.

3.5. Métricas clave

1. MRR: ingreso mensual recurrente generado por suscripciones activas.
2. Tasa de conversión Free → Pro: porcentaje de usuarios gratuitos que pasan a un plan de pago.
3. CAC y LTV: coste de adquisición de cliente y valor total esperado por cliente.
4. Churn mensual: porcentaje de usuarios o ingresos que se pierden cada mes.
5. Coste por 1.000 generaciones: gasto operativo asociado a producir mil outputs del modelo.

4. Estrategia corporativa (empresa existente)

4.1. Integración para ventaja competitiva

Una empresa existente puede integrar esta tecnología como capa de asistencia transversal en marketing, ventas y soporte.

Casos de impacto:

1. Marketing: borradores de emails y piezas para redes.
2. Ventas: propuestas y mensajes personalizados por cliente.
3. Soporte: respuestas base para consultas repetitivas.

4.2. Ruta de adopción recomendada

Fase 1 – Piloto (4–6 semanas)

1. Seleccionar un proceso concreto y acotado (por ejemplo, descripciones de producto).
2. Definir KPIs clave: tiempo por pieza, calidad percibida y ratio de aprobación.
3. Formar al equipo y establecer guías de uso operativas.

Fase 2 – Escalado (2–3 meses)

1. Ampliar la adopción a otros canales y equipos.
2. Integrar la solución con CRM, CMS o gestores de campañas.
3. Implementar gobernanza de prompts y versionado de plantillas.

Fase 3 – Optimización continua

1. Analizar el rendimiento por canal y tipo de contenido.
2. Ajustar prompts y estrategias de generación según resultados.
3. Incorporar paneles de auditoría y trazabilidad.

4.3. Ventaja competitiva concreta

1. Mayor velocidad operativa frente a alternativas del mercado.

2. Consistencia reforzada en el mensaje de marca.
3. Reducción del coste por pieza de contenido.
4. Capacidad de personalización a escala.

5. Convergencia tecnológica

5.1. IoT + IA

Escenario: fábrica equipada con sensores de temperatura, vibración, consumo energético y errores de línea.

Flujo operativo:

1. Los sensores IoT envían telemetría periódica.
2. Un servicio central agrega y normaliza los datos.
3. La IA convierte la información técnica en informes en lenguaje natural.
4. El responsable de operaciones recibe un resumen con recomendaciones.

Ejemplo de salida:

"La línea 3 presenta un incremento del 18% en vibración respecto a su media semanal. Se recomienda una inspección preventiva en las próximas 12 horas para evitar una parada no planificada."

Valor aportado:

1. Menor tiempo dedicado al análisis manual.
2. Decisiones operativas más rápidas y fundamentadas.
3. Comunicación más clara entre áreas técnicas y de negocio.

5.2. Blockchain + IA

Rol propuesto: garantizar la procedencia e integridad del contenido generado.

Flujo:

1. El texto se genera con metadatos (timestamp, versión del modelo, parámetros).

2. Se calcula un hash del contenido y sus metadatos.
3. El hash se registra en una Blockchain pública o permisionada.
4. Cualquier tercero puede verificar origen e integridad.

Casos de uso:

1. Auditoría de contenido regulado.
2. Prueba de autoría y fecha de generación.
3. Trazabilidad en procesos legales o financieros.

Precauciones:

1. No almacenar datos sensibles en la cadena.
2. Registrar únicamente el hash y referencias, nunca el contenido completo.

6. Viabilidad y hoja de ruta

6.1. Viabilidad técnica

1. Alta: arquitectura API + web + proxy ya operativa.
2. Escalable: separación clara entre frontend, backend y motor remoto.
3. Segura: la API key se gestiona en servidor, no en cliente.

6.2. Viabilidad de negocio

1. Media–Alta: creciente demanda de automatización de contenido.
2. Buen encaje en pymes y equipos con alta carga de redacción.
3. Monetización clara mediante modelo freemium/SaaS.

6.3. Roadmap sugerido

0–3 meses:

1. Producto mínimo estable.
2. Panel básico de uso y métricas.
3. Primeras integraciones (CMS/CRM simples).

3–6 meses:

1. Versión multiusuario con roles.
2. Plantillas específicas por sector.
3. Facturación y planes comerciales.

6–12 meses:

1. Módulo de reporting IoT para industria.
2. Trazabilidad mediante anclaje hash en Blockchain.
3. Verticalización por industria (retail, industria, servicios).

7. Conclusión

El proyecto demuestra que una IA generativa de texto puede evolucionar de un prototipo técnico a un activo de negocio real cuando se integra en procesos operativos. La combinación de automatización, modelo comercial adecuado y convergencia tecnológica permite construir un producto viable, escalable y con impacto medible.

La recomendación final es lanzar con un enfoque **freemium + SaaS**, priorizar casos de alto volumen de contenido y seguir una adopción empresarial gradual para capturar ventaja competitiva de forma sostenida.